

Security Review-GPT_01

Security-Review: Kryschuuu/ai-trading-firm

Stand: main , geprüft am 5. September 2026.

Ich habe insbesondere Auth/RBAC, Session-Handling, Broker-Control-Plane, Secret-Store, Rule-Engine, API-Routen, Audit-/Logging-Pfade, Deployment und Dependency-Versionen geprüft. Zusätzlich habe ich die aktuell veröffentlichten Security Advisories für relevante Dependencies abgeglichen.

<https://github.com/Kryschuuu/ai-trading-firm>

Gesamturteil

Für Paper-Trading: technisch bereits deutlich gehärtet, aber noch mehrere relevante Sicherheitslücken bzw. gefährliche Designkanten.

Für echtes Live-Trading: aktuell nicht freigabefähig.

Die wichtigsten Punkte sind nicht klassische SQL-Injection oder Kryptographiefehler, sondern **Authorization-/Trust-Boundary-Probleme und unnötig öffentliche Datenzugriffe**. Hinzu kommen zwei aktuelle Dependency-Probleme (next , ws).

Priorisierte Findings

Priorität	ID	Bereich	Bewertung
Kritisch	SEC-01	AuthN/AuthZ	Viewer kann unter bestimmter Konfiguration Admin-Session fälschen
Hoch	SEC-02	Datenexposition	Sensible Trading-/Audit-/Agentendaten über mehrere unauthentifizierte GET-APIs öffentlich
Hoch	SEC-03	Dependencies	next 16.3.1 liegt in aktuell verwundbaren Versionen
Hoch	SEC-04	Dependencies	ws ^8.18.0 erlaubt aktuell verwundbare Versionen
Mittel	SEC-05	AuthZ/Audit	Client kann Rollen-/Akteursattribution bei Rule-Änderungen fälschen
Mittel	SEC-06	AuthZ	Rule-Lifecycle-Aktionen sind nur durch firm.write , nicht durch eine spezifische Privilegstufe geschützt
Mittel	SEC-07	Secret Management	Bitunix fällt bei Control-Plane-Fehlern auf Env-Credentials zurück
Mittel	SEC-08	AuthN/AuthZ	Stateless Sessions besitzen keine unmittelbare Revocation und speichern Berechtigungen als Snapshot
Niedrig	SEC-09	Kryptographie	Secret-Memory-Hygiene ist in JS nur teilweise wirksam
Niedrig	SEC-10	Supply Chain	GitHub Actions werden nicht auf immutable Commit-SHAs gepinnt

SEC-01 — Privilege Escalation über signierte Session

Priorität: KRITISCH

Risiko: echtes Risiko, konditional exploitable

Der kritischste Befund steckt in authSession.ts .

Der Session-Signaturschlüssel ist optional unabhängig konfigurierbar:

```
FIRM_SESSION_SECRET
```

Fehlt dieser, wird der HMAC-Schlüssel deterministisch aus den konfigurierten Auth-Tokens abgeleitet. Dabei werden die vorhandenen Tokens einfach aneinandergereiht und SHA-256 darüber gebildet.

Gleichzeitig validiert verifySessionToken() zwar:

- HMAC,
- Payload-Schema,
- Ablauf,
- dass Permissions aus der erlaubten Permission-Liste stammen,

aber **nicht**, dass die permissions zur behaupteten Rolle gehören. Ebenso wird effectiveRole nicht erneut aus der aktuellen Auth-Konfiguration abgeleitet.

Das ist zusammen gefährlich.

Konkretes Angriffsszenario

Angenommen:

```
FIRM_VIEWER_TOKEN=<geheimer Viewer-Token>
FIRM_SESSION_SECRET=
FIRM_ADMIN_TOKEN=
FIRM_API_TOKEN=
```

Der Viewer kennt seinen eigenen Token zwangsläufig.

Damit kennt er das Material, aus dem `sessionSecret()` gebildet wird. Die Login-Route akzeptiert ausdrücklich auch Viewer-Credentials und stellt daraus eine signierte Session aus.

Da das Session-Payload anschließend beispielsweise

```
{
  "role": "admin",
  "effectiveRole": "admin",
  "permissions": [
    "firm.read",
    "firm.write",
    "firm.kill",
    "firm.config",
    "broker.credentials",
    "routing.modes.write",
    "live.gate"
  ]
}
```

enthalten dürfte, sofern es korrekt HMAC-signiert ist, fehlt serverseitig die entscheidende Relation:

| `role=admin` muss von einem tatsächlich adminberechtigten aktuellen Credential stammen.

Das wird nicht rekonstruiert.

Die Permission-Matrix selbst ist korrekt: `live.gate` ist ausschließlich Admin zugeordnet.

Das Problem liegt davor: **Ein gültig signiertes, aber vom Viewer selbst erzeugtes Admin-Payload wird als Autorität akzeptiert.**

Auswirkungen

Bei dieser Konfiguration kann ein Viewer potentiell:

- `firm.write`
- ``firm.kill`
- `firm.config`
- `broker.credentials`
- `routing.modes.write`
- `live.gate`

erlangen.

Das ist eine echte horizontale/vertikale Privilege-Escalation.

Remediation

Beste Lösung:

In Produktion `FIRM_SESSION_SECRET` verpflichtend machen und als unabhängiges Secret behandeln.

Noch wichtiger:

`sessionActor()` darf keine frei im Cookie gespeicherten Permissions als Autorität betrachten.

Beim Verifizieren:

```
const expectedPermissions = permissionsForRole(payload.effectiveRole);

if (!samePermissionSet(payload.permissions, expectedPermissions)) {
  return null;
}
```

Noch robuster:

- `role`
- ``effectiveRole`
- `elevated`
- `Permissions`

nicht vollständig als vertrauenswürdige Session-Autorität behandeln, sondern serverseitig neu ableiten.

Zusätzlich sollte eine Session ein `authEpoch` bzw. Credential-Version enthalten, damit Rotation/Revocation serverseitig greift.

Regression Test

Pflicht-Test:

| Nur `FIRM_VIEWER_TOKEN` gesetzt → Viewer darf niemals eine gültige Session mit `firm.write` oder `live.gate` erzeugen können.

SEC-02 — Sensible Daten sind über unauthentifizierte GET-APIs erreichbar

Priorität: HOCH

Risiko: echtes Risiko

Mehrere API-Routen dokumentieren selbst ausdrücklich, dass **kein Token für Lesezugriffe erforderlich ist**.

Besonders problematisch ist `/api/firm`.

Die Route liefert unter anderem:

- Agents
- Missions
- Positionen
- P&L
- Proposals
- Audit-Log
- Kill-Switch-Historie
- Agent-Messages
- Risk Limits
- Runtime-Konfiguration
- adaptive Risk-Daten
- Broker Registry
- Scheduler State
- Account-/Equity-Daten.

Noch problematischer sind die dedizierten Log-/Report-Routen.

`/api/firm/log` liefert rohe Agent-Messages inklusive:

```
content
meta
missionId
agentId
```

und rohe Audit-Details.

`/api/firm/report` liefert unter anderem:

- realisiertes P&L
- Drawdown
- Symbolstatistiken
- Entscheidungsverteilungen
- Audit-Ereignisse
- Recommendations
- Thesis
- Entry Zone
- Stop Loss
- Target
- Risk Flags.

Auch `/api/firm/rules` gibt das komplette Regelwerk, aktive Regeln, Feedback und Execution-Daten ohne vorgelagerte Authentifizierung zurück.

Zusätzlich exponieren `/api/providers` und `/api/routing` Modell-, Routing-, Budget-, Provider-, Health- und Entscheidungsinformationen ohne Token.

Warum das real problematisch ist

Das ist nicht nur „Dashboard-Information“.

Ein Remote-Angreifer kann damit:

- Handelsstrategie rekonstruieren,
- aktuelle Positionen und P&L beobachten,
- interne Risiko-Grenzen kennenlernen,
- Entscheidungslogik und Rules rekonstruieren,
- Agenten-Kommunikation analysieren,
- Betriebszustände überwachen,
- Audit-Ereignisse beobachten.

Bei einer später aktivierten Live-Broker-Integration wäre das noch wesentlich sensibler.

Verstärkender Deployment-Faktor

Der Service wird in der systemd-Vorlage normal als Netzwerkdienst betrieben; `next start` läuft auf Port 3369.

Die Anwendung selbst hat also keinen konzeptionellen „nur localhost“-Schutz für diese Daten-APIs.

Remediation

Für alle sensiblen READ-Endpunkte eine explizite Permission verlangen.

Beispiel:

```
const denied = requirePermission(req, "firm.read");
if (denied) return denied;
```

Für besonders sensible Ressourcen differenzieren:

```
firm.read
ops.view
strategy.read
audit.read
portfolio.read
broker.status
```

Mindestens:

- /api/firm
- /api/firm/log
- /api/firm/report
- /api/firm/rules
- /api/providers
- /api/routing

sollten nicht öffentlich sein.

Das Dashboard selbst kann weiterhin geschützt bleiben und serverseitig authentifiziert auf diese APIs zugreifen.

SEC-03 — Verwundbare Next.js-Version

Priorität: HOCH

Risiko: echtes Risiko; Exploitability hängt von Deployment ab

Im Projekt ist next mit

```
^16.3.1
```

deklariert. Die Lock-Datei ist vorhanden und enthält ebenfalls diese Dependency-Spezifikation.

Für Next.js wurde am **25. August 2026** eine **kritische ungepatchte RCE** gemeldet:

- betroffen: $\geq 16.0 < 16.3.3$
- behoben: 16.3.3
- CVSS 9.0
- betroffene Windows-Server können unauthentifizierte Remote Code Execution ermöglichen. ([GitHub](#))

Zusätzlich wurde für dieselbe Versionsgrenze eine weitere kritische RCE im Image Optimization API veröffentlicht:

- betroffen: $< 16.3.3$
- behoben: 16.3.3
- CVSS 9.5. ([GitHub](#))

Kontext des Projekts

Die bereitgestellte Deployment-Unit ist zwar eindeutig auf Linux/systemd ausgelegt.

Damit ist die Windows-RCE **für das dokumentierte Standarddeployment nicht unmittelbar reproduzierbar**.

Das Dependency-Level bleibt trotzdem ein Problem, weil:

1. package.json eine vulnerable Basisversion zulässt,
2. die Anwendung Next App Router verwendet,
3. die Release-Linie 16.3.1 nicht mehr auf dem sicheren Stand ist.

Remediation

Mindestens:

```
"next": "^16.3.3"
```

bzw. besser auf den aktuellen stabilen Patchstand aktualisieren.

Danach:

```
npm install next@latest
npm ci
npm audit
npm run typecheck
npm run lint
npm run security:live-gate
```

Für ein Security-Gate würde ich **nicht nur auf ^16.3.3 vertrauen**, sondern den tatsächlich aufgelösten Lockfile-Stand überwachen.

SEC-04 — ws erlaubt mehrere aktuell verwundbare Versionen

Priorität: HOCH

Risiko: wahrscheinlich echt, genaue Lock-Auflösung muss noch verifiziert werden

package.json verwendet:

```
ws: ^8.18.0
```

und damit eine Range, die mehrere verwundbare Versionen einschließt.

Für ws wurde 2026 eine High-Severity-Schwachstelle veröffentlicht:

CVE-2026-48779

Betroffen:

```
>= 8.0.0 < 8.21.0
```

Behoben:

```
8.21.0
```

Angriff: Netzwerk-Peer, keine Authentifizierung erforderlich, durch extrem viele kleine WebSocket-Fragmente kann Speicher erschöpft und der Prozess beendet werden. CVSS 7.5. ([GitHub](#))

Darüber hinaus:

CVE-2026-45736

Betroffen:

```
>= 8.0.0 < 8.20.1
```

Behoben:

```
8.20.1
```

Hier geht es um mögliche Offenlegung nicht initialisierten Speichers über eine bestimmte websocket.close()-Nutzung. ([GitHub](#))

Einschätzung

Die exakte im Lockfile installierte ws-Version konnte ich über die derzeit verfügbare GitHub-Dateiansicht nicht zuverlässig isolieren; deshalb behandle ich den Befund als **Dependency-Risiko**, nicht als bereits bewiesene Laufzeit-Exploitation.

Da das Projekt ws tatsächlich als Dependency führt, sollte das aber unmittelbar bereinigt werden.

Remediation

Mindestens:

```
"ws": "^8.21.0"
```

und Lockfile regenerieren.

Noch besser: aktuelles gepatchtes Release verwenden und danach:

```
npm ls ws
npm audit
```

als CI-Check erzwingen.

SEC-05 — Fälschbare Akteurs-/Rollenattribution bei Rule-Änderungen

Priorität: MITTEL

Risiko: echtes Audit-Integrity-Problem

Bei `/api/firm/rules/[id]` wird die Authentifizierung nur über `guardWrite(req)` durchgeführt.

Danach wird ein vom Client gelieferter Wert übernommen:

```
by?: string
```

und an die Rule-Service-Funktionen weitergereicht:

```
activateRule(id, body.by ?? "API")
pauseRule(id, body.by ?? "API")
archiveRule(id, body.by ?? "API")
rollbackRule(id, body.by ?? "API")
rejectRule(id, ..., body.by ?? "API")
```

Das Service-Modul schreibt diesen `by`-Wert anschließend als Audit-Attribution.

Damit kann beispielsweise ein Operator im Request angeben:

```
{
  "action": "activate",
  "by": "ADMIN"
}
```

Die tatsächliche Authentität des Requests bleibt davon unberührt.

Auswirkungen

Der Angreifer erhält dadurch keine zusätzlichen Rechte, aber der Audit-Trail wird:

- semantisch unzuverlässig,
- forensisch schwächer,
- für Incident Response manipulierbar.

Bei einem Trading-System ist genau das problematisch, weil später nicht mehr eindeutig feststellbar ist, **wer eine Strategieänderung wirklich ausgelöst hat**.

Remediation

`by` vollständig aus dem externen API-Contract entfernen.

Stattdessen:

```
const actor = resolveAuth(req).actor;
const actorId = actor.auditId;

await activateRule(id, actorId);
```

Auch `sourceRole` sollte nicht aus dem Client übernommen werden.

SEC-06 — Rule-Lifecycle benötigt keine spezifische Privilegstufe

Priorität: MITTEL

Risiko: echtes Autorisierungsproblem, falls Operator nicht strategische Governance besitzen soll

Die Rollenmatrix enthält:

```
operator:
  firm.write
  firm.kill
  firm.config
  broker.test
```

Admin erhält zusätzlich `broker.credentials`, `routing.modes.write` und `live.gate`.

Die Rule-APIs verlangen jedoch lediglich:

```
guardWrite(req)
```

und lassen damit einen Benutzer mit `firm.write` unter anderem:

- Regeln anlegen,
- Regeln aktivieren,
- Regeln pausieren,
- Regeln archivieren,
- Regeln zurückrollen,
- Regeln ablehnen.

Das widerspricht zumindest teilweise der im Rule-Service beschriebenen Governance-Idee, wonach Aktivierung eine explizite, auditierte Handlung sein soll.

Remediation

Eine eigene Permission einführen:

```
strategy.rules.write
strategy.rules.activate
strategy.rules.rollback
```

Beispielsweise:

```
viewer -> read
operator -> create/edit/pause
admin -> activate/rollback/archive
```

Noch besser: Aktivierung einer Regel und insbesondere Rollback nur mit einem expliziten Governance-Gate.

SEC-07 — Secret-Store fällt bei Fehlern auf Env-Credentials zurück

Priorität: MITTEL

Risiko: echter Trust-Boundary-Bruch

Der zentrale Control-Plane-Secret-Store ist AES-256-GCM-basiert und grundsätzlich sauber aufgebaut.

Aber die Bridge für Bitunix besitzt einen sicherheitsrelevanten Fallback:

```
Control Plane credential store
  ↓ Fehler / kein Datensatz
Environment credentials
```

`createVenueBackedNamedStore()` fängt einen Fehler des verschlüsselten Stores ab und verwendet anschließend den Env-Fallback.

Das wird im Bitunix-Adapter ausdrücklich produktiv verwendet:

```
createVenueBackedNamedStore(...)
envFallback = BITUNIX_API_KEY / BITUNIX_API_SECRET
```

Problem

Der Control Plane kann damit sagen:

```
configured = false
```

während der tatsächliche Broker-Adapter weiterhin Credentials aus:

```
BITUNIX_API_KEY
BITUNIX_API_SECRET
```

verwenden kann.

Noch problematischer:

Ein `AUTH_FAILED` bzw. korruptes Envelope wird nicht zwingend zu einem harten Credential-Stop, sondern kann in den Legacy-Fallback laufen.

Das erzeugt zwei Wahrheiten:

| Control Plane ≠ tatsächliche Broker-Credentials.

Remediation

Produktiv:

kein Credential-Fallback nach Store-Fehlern.

Zulässige Semantik:

```
credential exists in secure store → use it
credential absent → no credential
store failure → HARD FAIL
```

Env-Credentials sollten nur in einem expliziten `development/test` -Modus erlaubt sein.

SEC-08 — Sessions sind nicht sofort widerrufbar

Priorität: MITTEL

Risiko: echtes Lifecycle-/Revocation-Problem

Sessions sind stateless, HMAC-signiert und haben 15 Minuten TTL.

Das ist grundsätzlich ein gutes Design.

Aber der signierte Session-Payload enthält:

```
role
effectiveRole
elevated
permissions
```

und diese Werte werden bei jeder Anfrage aus der Session übernommen.

Damit kann eine Session bis zu 15 Minuten weiter gültig bleiben, obwohl:

- ein Token rotiert wurde,
- Berechtigungen geändert wurden,
- eine Rolle reduziert wurde,
- ein Operator degradiert wurde.

Ist ein separates `FIRM_SESSION_SECRET` gesetzt, ist eine Token-Rotation sogar vollständig von laufenden Sessions entkoppelt.

Remediation

Eine serverseitige Session-Revocation-Epoche einführen:

```
session.authEpoch = currentAuthEpoch
```

und bei Credential-Rotation:

```
authEpoch++
```

Alternativ Sessions vollständig serverseitig speichern.

Für das Trading-System halte ich **maximal 5 Minuten** Session-TTL plus Revocation-Epoche für angemessener.

SEC-09 — Memory-Hygiene schützt JS-Strings nicht wirklich

Priorität: NIEDRIG

Risiko: kein Remote-Exploit, aber Kryptographie-/Secret-Hardening-Thema

Der Secret Store bemüht sich ausdrücklich um `Buffer` -basierte Secret-Verarbeitung und `zeroize()`. Das ist positiv.

Aber:

```
plaintext.toString("utf8")
```

erzeugt einen JavaScript-String, und anschließend:

```
return {
  apiKey: parsed.apiKey,
  apiSecret: parsed.apiSecret
}
```

Diese Strings sind immutable und können nicht deterministisch überschrieben werden.

Dasselbe gilt für Credentials, die danach im normalen JS-Heap existieren.

Einschätzung

Kein sinnvoller Remote-Angriffspfad.

Relevant wird es bei:

- Heap-Dump,
- Crash-Dumps,
- Debugging,
- Process Compromise,
- forensischem Speicherzugriff.

Remediation

Die vorhandene Buffer-Hygiene beibehalten, aber die Behauptung „Klartext existiert nicht als langlebiger String“ aus der Dokumentation entfernen.

Zusätzlich:

- Heap Dumps deaktivieren,
- Debug Inspector nicht exponieren,
- Core Dumps minimieren,
- Credentials möglichst kurzlebig halten.

SEC-10 — GitHub Actions nicht auf immutable SHAs gepinnt

Priorität: NIEDRIG

Risiko: Supply-Chain-Hardening, aktuell kein direkter Exploit nachgewiesen

Die CI verwendet:

```
uses: actions/checkout@v4
uses: actions/setup-node@v4
uses: actions/upload-artifact@v4
```

Tags wie @v4 sind mutable Referenzen. Ein kompromittiertes bzw. nachträglich verschobenes Tag könnte theoretisch Code in der CI ausführen.

Positiv ist:

```
permissions:
  contents: read
```

Die Workflows haben also bereits eine sinnvolle minimale Token-Berechtigung.

Remediation

Actions auf Commit-SHAs pinnen:

```
uses: actions/checkout@<vollständige-Commit-SHA>
```

und Renovate/Dependabot nutzen, um diese SHAs kontrolliert zu aktualisieren.

Injection-Review

SQL / Command / URL Injection

Kein bestätigter kritischer Injection-Befund im geprüften Code.

Die Rule-Engine macht hier einiges richtig:

- Whitelist der Felder
- Whitelist der Operatoren
- strikte Typvalidierung
- Symbolnormalisierung
- numerische Begrenzungen
- keine dynamischen Funktionen
- keine ausführbaren LLM-Strings.

Bei den DB-Abfragen wird Drizzle verwendet; beispielsweise werden Filterwerte über `eq()` gebunden und nicht als SQL-String konkatenierter User Input eingesetzt.

Auch der Market-Data-Code verwendet für externe Symbolwerte `encodeURIComponent()` und Whitelists für Intervalle.

Dependency-Kontext

Das ist wichtig, weil Drizzle selbst 2026 eine SQL-Injection-Schwachstelle hatte:

- betroffen: <=0.45.1
- behoben: 0.45.2
- CVE-2026-39356. ([GitHub](#))

Das Projekt verwendet bereits:

```
drizzle-orm 0.45.2
```

Daher ist dieser konkrete Drizzle-Befund **kein Finding gegen das Projekt**.

Kryptographie-Review

Hier ist der Code insgesamt deutlich besser als bei AuthZ:

Positiv

Der Secret Store verwendet:

```
AES-256-GCM
12-byte random IV
16-byte authentication tag
AAD = Venue
```

mit `randomBytes()` und Authenticated Encryption.

Die Venue-Bindung über AAD ist sinnvoll: ein Ciphertext kann nicht einfach auf eine andere Venue umgehängt werden, ohne dass die GCM-Authentifizierung scheitert.

Auch die Dateiablage erzwingt:

```
0600
```

und validiert Venue-Namen gegen Pfad-Traversal.

Hauptproblem

Nicht AES ist das Problem, sondern **Key Governance**:

```
SECRET_STORE_KEY
```

ist derzeit das zentrale Master Secret aus der Umgebung. Der vorgesehene AWS-KMS-Pfad ist noch nicht implementiert.

Für einen Single-Node-Paper-Trader ist das vertretbar.

Für echtes Trading würde ich verlangen:

```
KMS/HSM
→ envelope encryption
→ key versioning
→ rotation
→ auditierter key lifecycle
```

Abhängigkeiten: aktuell besonders relevant

Die Dependency-Situation würde ich im aktuellen Stand so bewerten:

Package	Projektstand	Sicherheitsstatus
next	^16.3.1	Upgrade erforderlich ; 16.3.1 liegt unter 16.3.3
ws	^8.18.0	Upgrade erforderlich auf >=8.21.0
drizzle-orm	0.45.2	gut; aktueller SQLi-Fix enthalten
react	19.2.6	der geprüfte RSC-DoS-Fix ist enthalten
pg	8.20.0	kein vergleichbarer aktueller High/Critical-Fund in dieser Prüfung bestätigt
react-markdown	^10.1.0	kein konkreter Security-Fund in diesem Review bestätigt

Für React ist beispielsweise die im Mai 2026 behobene RSC-DoS-Lücke bis 19.2.5 betroffen und ab 19.2.6 behoben. Das Projekt liegt hier bereits auf der sicheren Version. ([GitHub](#))

Was bereits gut umgesetzt ist

Die aktuelle Version enthält eine Reihe von Maßnahmen, die ich ausdrücklich **nicht** als Findings einstufen würde:

Kill-Switch

Der Disarm ist wesentlich besser geschützt:

```
ADMIN / live.gate
+ CSRF
+ single-use nonce
+ 60 Sekunden TTL
```

und der Nonce wird synchron verbraucht.

Das ist eine sinnvolle Trennung zwischen:

```
Operator darf STOP auslösen
Admin darf STOP wieder aufheben
```

Secret Storage

AES-GCM, AAD, random IV, keine Klartextablage und Dateiberechtigungen sind vernünftig umgesetzt.

Rule Engine

Die Idee

```
LLM → sanitize → whitelist → clamp → deterministic executor
```

ist sicherheitstechnisch deutlich besser als LLM-generierte ausführbare Logik.

CSRF

Der Session-Pfad benutzt:

```
HttpOnly firm_session
+
non-HttpOnly firm_csrf
+
SameSite=Strict
+
Custom Header
```

Das ist ein brauchbares Double-Submit-Konzept.

Empfohlene Reihenfolge der Behebung

1. Sofort

SEC-01: Session-Signierung reparieren.

Die Anwendung darf niemals erlauben, dass ein Low-Privilege-Credential gleichzeitig der Schlüssel zur Herstellung einer High-Privilege-Session ist.

SEC-03 / SEC-04: next und ws aktualisieren.

2. Vor weiterem Live-Ausbau

SEC-02: Alle sensitiven GET-APIs authentifizieren.

SEC-07: Env-Credential-Fallback aus dem produktiven Control-Plane-Pfad entfernen.

3. Vor echter Multi-Role-Nutzung

SEC-05 / SEC-06: Rollen- und Audit-Attribution ausschließlich serverseitig aus dem authentifizierten Actor ableiten und Rule-Lifecycle mit eigenen Permissions versehen.

4. Danach

SEC-08: Session-Revocation implementieren.

SEC-09: Memory-Hygiene-Dokumentation korrigieren.

SEC-10: GitHub Actions auf Commit-SHAs pinnen.

Finales Security Rating

Bereich	Urteil
Injection	gut / kein bestätigter kritischer Befund
Authentication	kritischer Designfehler in bestimmter Token-Konfiguration
Authorization	mehrere mittlere Trust-/Governance-Probleme
Data Exposure	hoch problematisch
Secrets at Rest	gut, aber Fallback-Problem
Kryptographie	Primitive gut, Key Management ausbaufähig
Dependencies	aktuell nicht sauber gepatcht
CI/Supply Chain	brauchbar, aber nicht maximal gehärtet
Paper-Trading	bedingt einsatzfähig
Live-Trading	nicht freigabefähig

Der **wichtigste technische Fix ist SEC-01**. Der ist deutlich gravierender als die klassischen Scanner-Funde: Ein System kann sehr viele Security-Header, CSRF-Checks und AES-256-GCM besitzen und trotzdem eine fundamentale Privilege-Escalation haben, wenn **die Session selbst zur Autorität wird und die Rollen nicht mehr an das tatsächlich authentifizierte Credential gebunden sind**.

Die Dependency-Bewertung ist ebenfalls zeitkritisch: `next` 16.3.1 liegt unter dem am **25. August 2026** veröffentlichten Security-Fix 16.3.3, und `ws` 8.18.x liegt in der betroffenen Range `<8.21.0`. ([GitHub](#))